Министерство науки и высшего образования Российской Федерации

федеральное государственное бюджетное образовательное учреждение
высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
МОРДОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
им. Н.П. ОГАРЁВА»
(ФГБОУ ВО «МГУ им. Н.П. Огарёва»)

Факультет математики и информационных технологий

Кафедра анализа данных и искусственного интеллекта

ОЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ «АВТОМАТИЗИРОВАННОЕ ФУНКЦИОНАЛЬНОЕ ТЕСТИРОВАНИЕ ВЕБ-САЙТА С ПОМОЩЬЮ ФРЕЙМВОРКА SELENIUM»

по дисциплине: Методы тестирования программных продуктов

Автор отчёта _____ Т.С. Бурлакова
подпись, дата

Обозначение отчёта ЛР–02069964–02.04.02–02–26

Направление подготовки 02.04.02 Фундаментальная информатика и информационные технологии

Профиль Информатика и компьютерные науки

Преподаватель,
канд. физ.-мат. наук _____ А.В. Попов
подпись, дата

Саранск 2026

1 Описание предмета тестирования

В связи с невозможностью проведения автоматического тестирования веб-сайта интернет-магазина парфюмерии и косметики «Золотое яблоко» (URL: https://goldapple.ru/), для которого проводилось ручное тестирование, было принято решение сменить предмет тестирования.

В качестве предмета тестирования в рамках данной лабораторной работы выступает веб-сайт службы доставки японской еды «Суши Сет» (URL: https://saransk.setsushi.ru/), предназначенный для онлайн-заказа блюд японской кухни с доставкой в городе Саранск. Сайт предоставляет пользователю следующие основные функции: просмотр ассортимента блюд по категориям (новинки, роллы, сеты, шаурма), изучение состава и веса каждого блюда, добавление товаров в корзину с главной страницы или из карточки товара, изменение количества и удаление позиций в корзине с автоматическим пересчетом итоговой суммы, а также оформление заказа с выбором способа оплаты и вводом контактных данных для доставки.

На рисунке 1 представлена главная страница сайта.

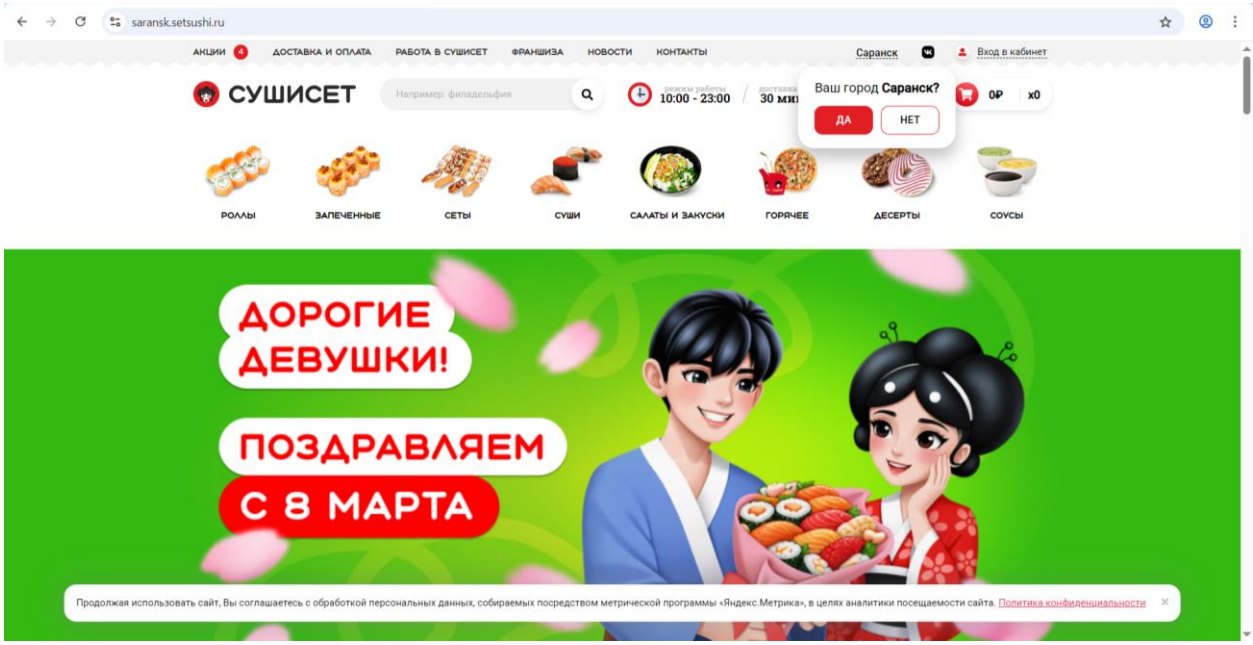

Рисунок 1 – Главная страница сайта

2 Описание окружения тестирования

Тестирование осуществлялось на компьютере со следующими характеристиками.

Процессор: 12th Gen Intel(R) Core(TM) i3-1215U, 1200 МГц, ядер: 6, логических процессоров: 8.

Видеокарта: Itnel(R) UHD Graphics.

Оперативная память: 8 ГБ, 3200 МГц.

Тип системы: 64-разрядная операционная система, процессор x64.

Операционная система: Microsoft Windows 10 Pro.

Разрешение экрана: 1920 x 1080.

Браузер: Google Chrome (145.0.7632.160); расширения браузера не использовались.

Антивирус: Kaspersky Standard.

3 Сценарии использования и описание автоматизированного тестирования

Настройка тестового окружения представлена в листинге 1: здесь подключаются необходимые библиотеки, настраиваются параметры браузера (отключение уведомлений), инициализируется WebDriver, создаётся директория для скриншотов и определяются вспомогательные функции для работы с элементами на странице.

Листинг 1 – Настройка тестового окружения

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options
from datetime import datetime
import random
import time
import os

chrome_options = Options()
prefs = {
    "profile.default_content_setting_values.notifications": 2
}
chrome_options.add_experimental_option("prefs", prefs)
chrome_options.add_argument("--disable-notifications")
chrome_options.add_argument("--disable-popup-blocking")

driver = webdriver.Chrome(options=chrome_options)
driver.maximize_window()
wait = WebDriverWait(driver, 20)

if not os.path.exists("screenshots"):
    os.makedirs("screenshots")

def take_screenshot(step_name: str) -> None:
    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    file_path = f"screenshots/{timestamp}_{step_name}.png"
    driver.save_screenshot(file_path)
    print(f"   Скриншот сохранен: {file_path}")

def scrollTo(xPath: str) -> None:
    item    =    wait.until(EC.presence_of_element_located((By.XPATH,
xPath)))
    driver.execute_script("arguments[0].scrollIntoView({behavior:
'smooth', block: 'center'});", item)
    time.sleep(2)
```

```
def clickTo(xPath: str) -> None:
    item = wait.until(EC.element_to_be_clickable((By.XPATH, xPath)))
    driver.execute_script("arguments[0].click()", item)
    time.sleep(2)
```

Позитивный сценарий №1: поиск товара.

Цель пользователя: найти блюдо «Удон с креветками» через поисковую строку.

Предусловие: неавторизованный пользователь находится на главной странице сайта.

Последовательность действий:

1. Ввести в поисковую строку в шапке сайта часть названия блюда (например, «удон»).

2. В предлагаемом выпадающем списке товаров выбрать «Удон с креветками».

3. Перейти на страницу с карточкой выбранного товара.

Ожидаемый результат: откроется корректная карточка товара.

Код для автоматического тестирования данного кейса представлен в листинге 2.

Листинг 2 – Автоматическое тестирование позитивного кейса №1

```
def testCase1() -> None:
    print("Позитивный кейс №1: поиск товара")
    driver.get("https://saransk.setsushi.ru/")
    print("1. Сайт открыт")
    time.sleep(3)
    take_screenshot("site_opened")

    print("2. Ввод текста 'удон' в поисковую строку")
    search_input = wait.until(EC.element_to_be_clickable(
            (By.XPATH,                          "//form[contains(@class,
'header_main_search')]//input[@placeholder='Например: филадельфия']")
        ))
    search_input.click()
    search_input.clear()
    search_input.send_keys("удон")
    print("   Текст 'удон' введен")
    take_screenshot("search_text_entered")
    time.sleep(5)

    print("3. Выбор 'Удон с креветками' в выпадающем списке")
```

```
    udon_item = wait.until(EC.element_to_be_clickable(
            (By.XPATH, "//*[contains(text(), 'Удон с креветками')]")
        ))
    udon_item.click()
    print("   'Удон с креветками' выбран из выпадающего списка")
    take_screenshot("udon_selected_from_dropdown")
    time.sleep(5)

    print("4. Загружена карточка товара")
    take_screenshot("final_page")

    print("5. Тест успешно завершен")
    print()
```

Позитивный сценарий №2: выбор блюда в каталоге и добавление товара в корзину.

Цель пользователя: найти блюдо «Запеченный лава с курицей» в каталоге товаров и добавить его в корзину.

Предусловие: неавторизованный пользователь находится на главной странице сайта.

Последовательность действий:

1. Перейти во вкладку «Запеченные роллы».

2. Найти и выбрать ролл «Запеченный лава с курицей».

3. Нажать на название товара, чтобы перейти в его карточку.

4. Нажать на кнопку «В корзину».

5. В открывшемся окне указать способ доставки «Самовывоз» и выбрать любой адрес из доступных.

6. Нажать на кнопку «Выбрать», чтобы применить адрес самовывоза.

8. Нажать на иконку корзины в правом верхнем углу, чтобы проверить содержимое.

Ожидаемый результат: товар появился в корзине с корректно указанным названием, ценой и количеством.

Код для автоматического тестирования данного кейса представлен в листинге 3.

Листинг 3 – Автоматическое тестирование позитивного кейса №2

```
def testCase2() -> None:
    print("Позитивный кейс №2: добавление товара в корзину")
    driver.get("https://saransk.setsushi.ru/")
    print("1. Сайт открыт")
    time.sleep(3)
    take_screenshot("site_opened")

    print("2. Переходим в раздел 'Запеченные роллы'")
    clickTo("//*[contains(text(),          'Запеченные')]          |
//*[contains(text(),   'Запеченные   роллы')   or   contains(text(),
'Запеченные')] | //a[contains(@href, 'baked')]")
    take_screenshot("baked_rolls_category")
    time.sleep(3)

    print("3. Находим и выбираем ролл 'Запеченный лава с курицей'")
    scrollTo("//p[contains(text(), 'Запеченный лава с курицей')]")
    take_screenshot("roll_found")

    print("4. Нажимаем на название ролла, чтобы открыть карточку
товара")
    clickTo("//p[contains(text(), 'Запеченный лава с курицей')]")
    take_screenshot("product_card_opened")

    print("5. Нажимаем на кнопку 'В корзину'")
    scrollTo("//button[contains(text(), 'В корзину')]")
    clickTo("//button[contains(text(), 'В корзину')]")
    take_screenshot("add_to_cart_clicked")

    print("6. Выбираем самовывоз")
    clickTo("//*[contains(text(), 'Самовывоз')]")
    take_screenshot("pickup_selected")

    print("7. Выбираем первый доступный адрес")
    clickTo("(//div[contains(@class, 'self-item')])[1]")
    take_screenshot("address_selected")

    print("8. Подтверждаем выбор адреса")
    clickTo("//button[contains(text(), 'Выбрать')]")
    take_screenshot("confirm_selected")
    time.sleep(3)

    print("9. Переходим в корзину")
    clickTo("//button[contains(text(),         'Корзина')]         |
//a[contains(text(),    'Корзина')]    |    //button[contains(@class,
'header_main_cart_button')]")
    time.sleep(3)
    take_screenshot("cart_opened")

    print("10. Тест успешно завершен")
    print()
```

Негативный сценарий №1: поиск товара с некорректным названием.

Цель пользователя: найти товар, но ввести некорректный поисковый запрос.

Предусловие: пользователь находится на главной странице сайта.

Последовательность действий:

1. Ввести в поисковую строку в шапке сайта запрос, состоящий из специальных символов без букв (например, «@#$%^&*»).

2. Нажать кнопку «Найти» или клавишу Enter.

Ожидаемый результат: в выпадающем списке с результатами поиска должно появиться сообщение: «Нет результатов» или «Ничего не найдено».

Код для автоматического тестирования данного кейса представлен в листинге 4.

Листинг 4 – Автоматическое тестирование негативного кейса №1

```
def testCase3() -> None:
    print("Негативный кейс №1: поиск товара с некорректным названием")
    driver.get("https://saransk.setsushi.ru/")
    print("1. Сайт открыт")
    time.sleep(3)
    take_screenshot("site_opened")

    print("2. Ввод текста '@#$%^&*' в поисковую строку")
    search_input = wait.until(EC.element_to_be_clickable(
            (By.XPATH,                        "//form[contains(@class,
'header_main_search')]//input[@placeholder='Например: филадельфия']")
        ))
    search_input.click()
    search_input.clear()
    search_input.send_keys("@#$%^&*")
    print("   Текст '@#$%^&*' введен")
    take_screenshot("search_text_entered")
    time.sleep(5)

    # Проверяем наличие выпадающего списка
    dropdown = driver.find_element(By.XPATH,
        "//div[contains(@class, 'search')] | " +
        "//div[contains(@class, 'search-results')] | " +
        "//ul[contains(@class, 'search')]"
    )

    # Ищем все элементы внутри выпадающего списка
    dropdown_items = dropdown.find_elements(By.XPATH,
```

```
        ".//li | .//div[contains(@class, 'item')] | .//a"
    )

    # Проверяем, пустой ли список
    if len(dropdown_items) == 0:
        print(f"   Выпадающий список пустой (нет результатов)")
        take_screenshot("dropdown_empty_correct")
        print("3. Тест успешно завершен")
    else:
        print(f"    Выпадающий список НЕ пустой! Найдено элементов:
{len(dropdown_items)}")
        take_screenshot("dropdown_not_empty_error")
        print("3. Тест не пройден")

    print()
```

4 Результаты автоматического тестирования

На рисунке 2 представлен вывод логов по результатам выполненного автоматического тестирования.

```
C:\Users\Home\Desktop>python Task03.py
Позитивный кейс №1: поиск товара
1. Сайт открыт
   Скриншот сохранен: screenshots/2026-03-09_16-03-32_site_opened.png
2. Ввод текста 'удон' в поисковую строку
   Текст 'удон' введен
   Скриншот сохранен: screenshots/2026-03-09_16-03-32_search_text_entered.png
3. Выбор 'Удон с креветками' в выпадающем списке
   'Удон с креветками' выбран из выпадающего списка
   Скриншот сохранен: screenshots/2026-03-09_16-03-38_udon_selected_from_dropdown.png
4. Загружена карточка товара
   Скриншот сохранен: screenshots/2026-03-09_16-03-43_final_page.png
5. Тест успешно завершен

Позитивный кейс №2: добавление товара в корзину
1. Сайт открыт
   Скриншот сохранен: screenshots/2026-03-09_16-03-48_site_opened.png
2. Переходим в раздел 'Запеченные роллы'
   Скриншот сохранен: screenshots/2026-03-09_16-03-50_baked_rolls_category.png
3. Находим и выбираем ролл 'Запеченный лава с курицей'
   Скриншот сохранен: screenshots/2026-03-09_16-03-56_roll_found.png
4. Нажимаем на название ролла, чтобы открыть карточку товара
   Скриншот сохранен: screenshots/2026-03-09_16-03-58_product_card_opened.png
5. Нажимаем на кнопку 'В корзину'
   Скриншот сохранен: screenshots/2026-03-09_16-04-02_add_to_cart_clicked.png
6. Выбираем самовывоз
   Скриншот сохранен: screenshots/2026-03-09_16-04-05_pickup_selected.png
7. Выбираем первый доступный адрес
   Скриншот сохранен: screenshots/2026-03-09_16-04-07_address_selected.png
8. Подтверждаем выбор адреса
   Скриншот сохранен: screenshots/2026-03-09_16-04-09_confirm_selected.png
9. Переходим в корзину
   Скриншот сохранен: screenshots/2026-03-09_16-04-17_cart_opened.png
10. Тест успешно завершен

Негативный кейс №1: поиск товара с некорректным названием
1. Сайт открыт
   Скриншот сохранен: screenshots/2026-03-09_16-04-22_site_opened.png
2. Ввод текста '@#$%^&*' в поисковую строку
   Текст '@#$%^&*' введен
   Скриншот сохранен: screenshots/2026-03-09_16-04-23_search_text_entered.png
   Выпадающий список пустой (нет результатов)
   Скриншот сохранен: screenshots/2026-03-09_16-04-28_dropdown_empty_correct.png
3. Тест успешно завершен
```

Рисунок 2 – Логи выполнения автоматического тестирования

Позитивный сценарий №1: поиск товара.

На рисунках 3 – 5 представлены результаты автоматического тестирования позитивного кейса №1.

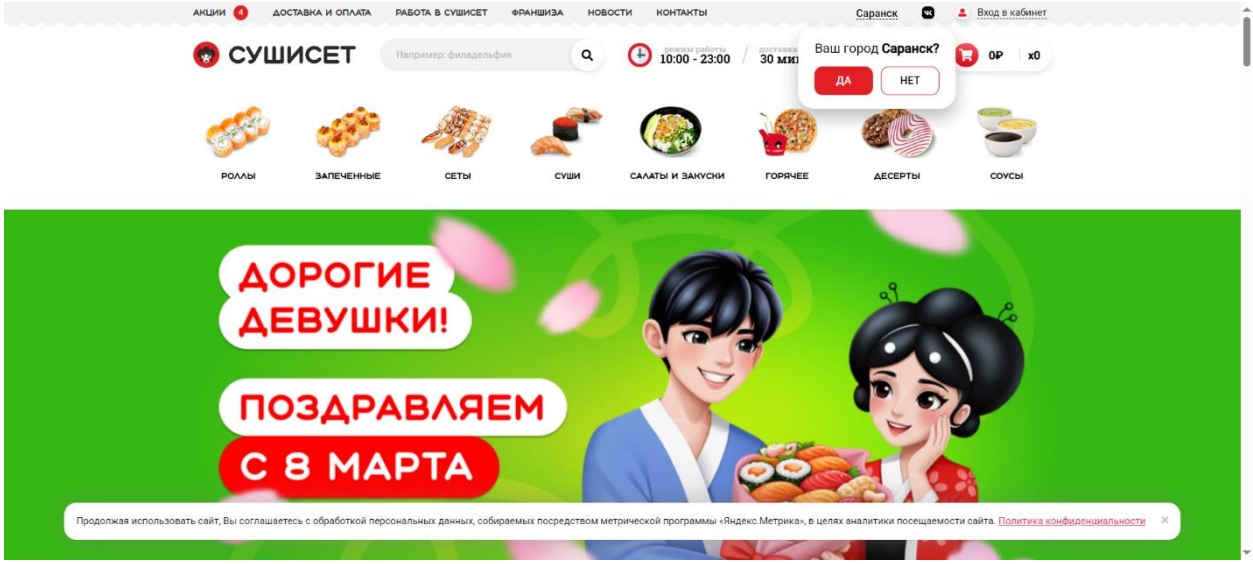

Рисунок 3 – Открыта главная страница сайта

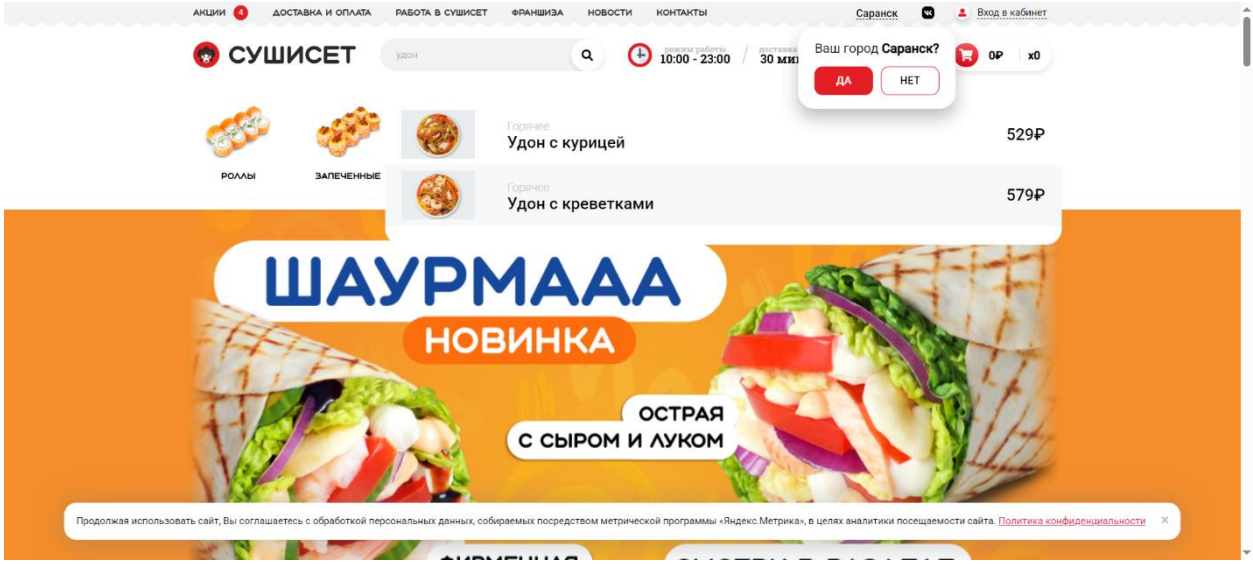

Рисунок 4 – Ввод в поисковую строку слова «удон»

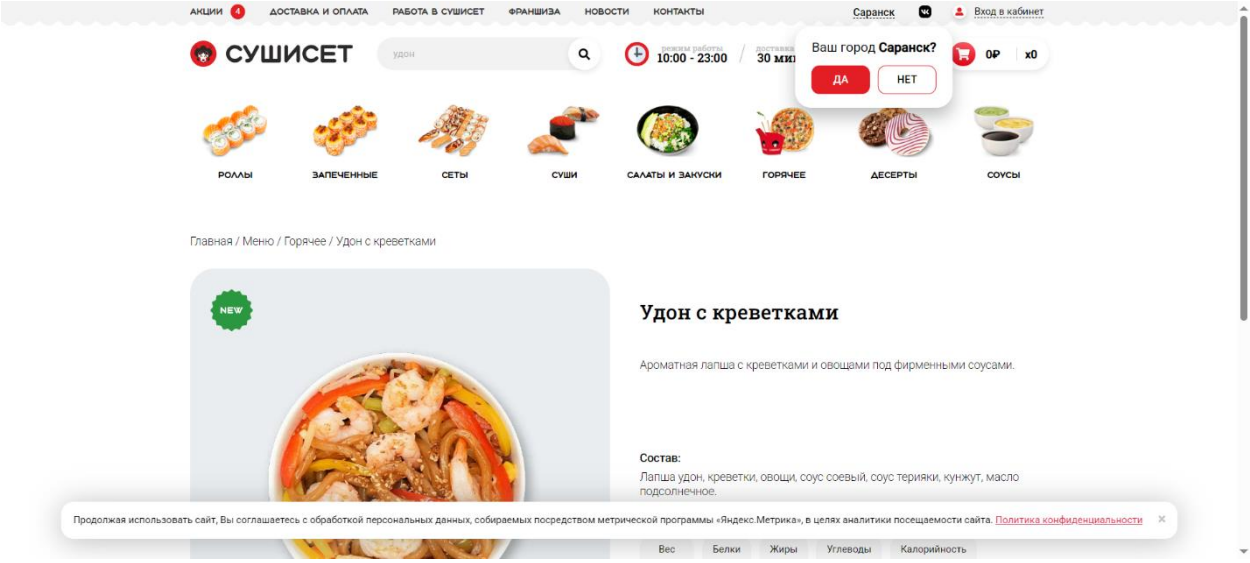

Рисунок 5 – Переход в карточку товара «Удон с креветками»

Фактический результат полностью совпал с ожидаемым: открыта карточка товара «Удон с креветками».

Позитивный сценарий №2: выбор блюда в каталоге и добавление товара в корзину.

На рисунках 6 – 13 представлены результаты автоматического тестирования позитивного кейса №2.

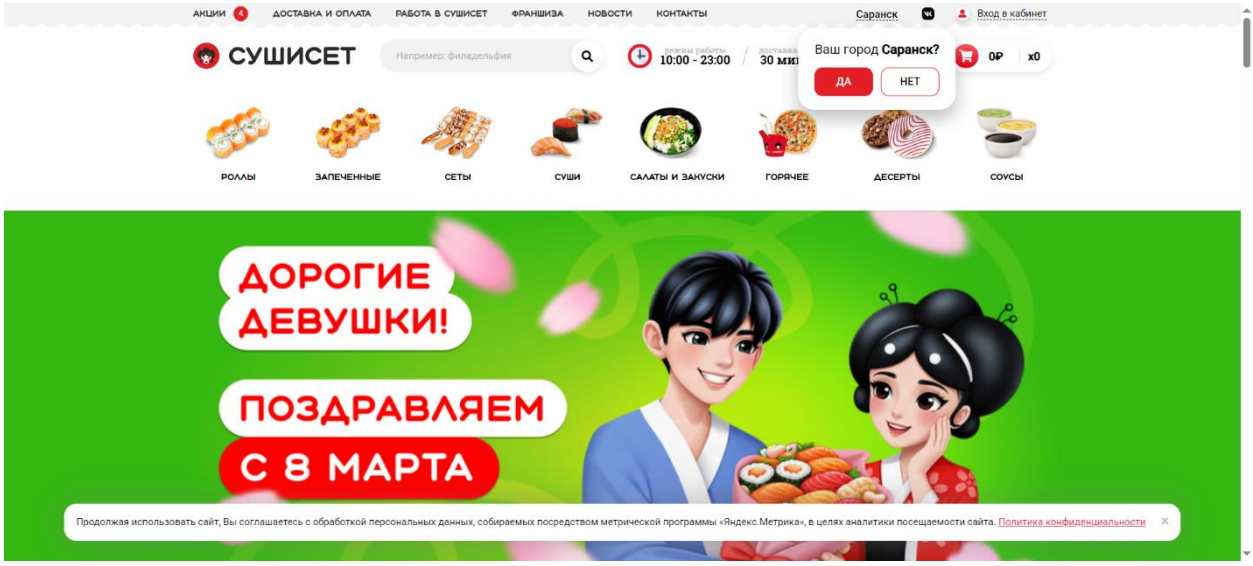

Рисунок 6 – Открыта главная страница сайта

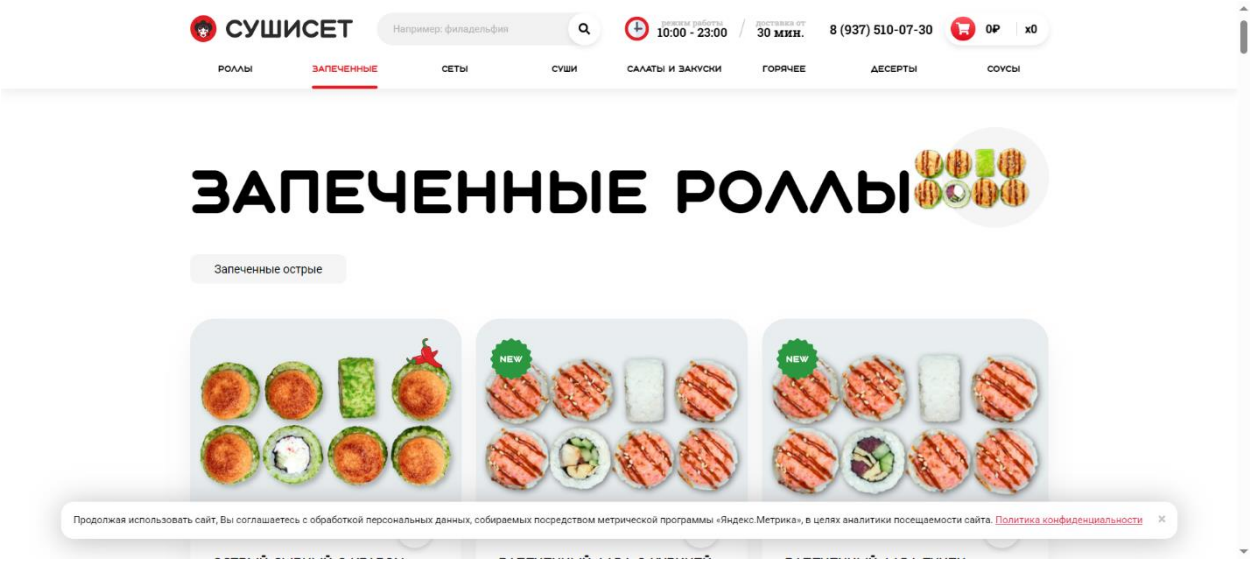

Рисунок 7 – Переход в раздел «Запеченные роллы»

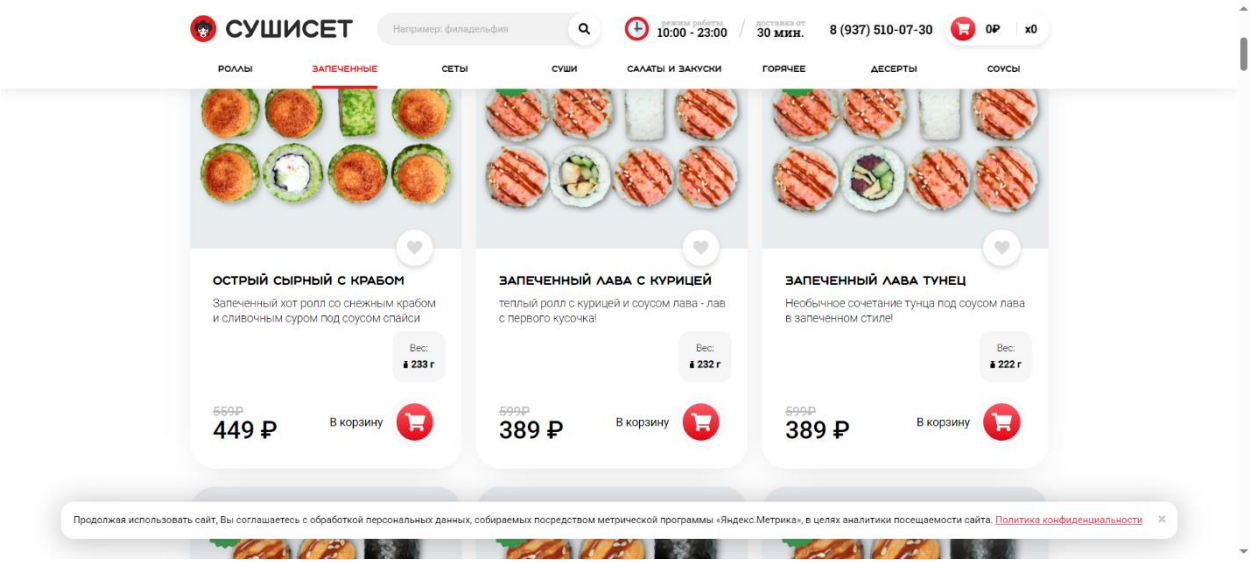

Рисунок 8 – Поиск ролла «Запеченный лава с курицей»

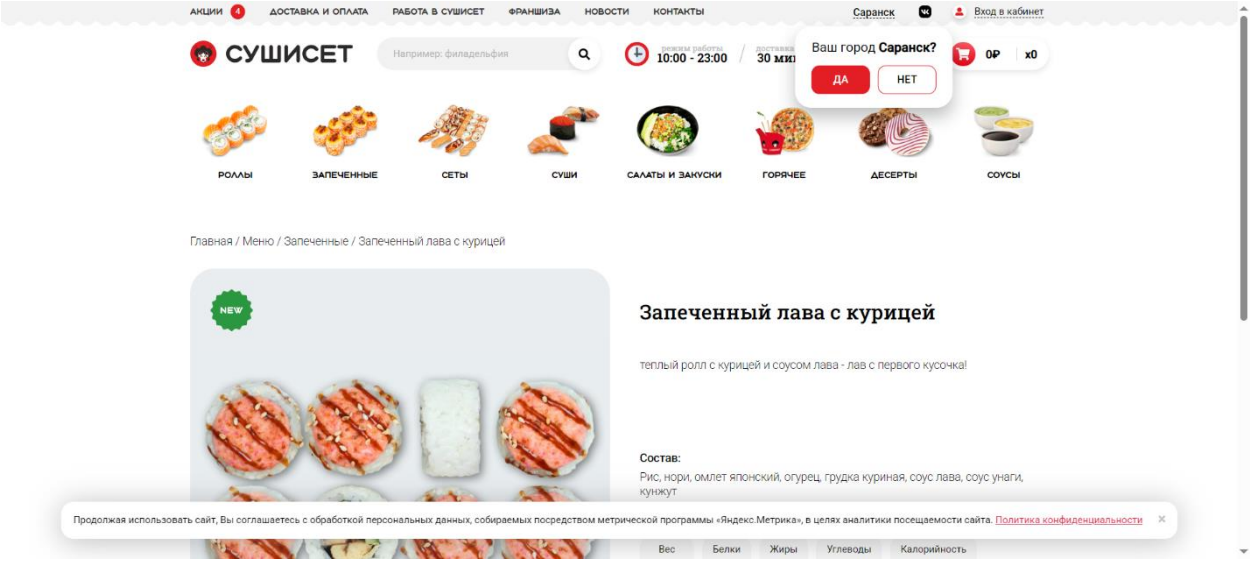

Рисунок 9 – Переход в карточку товара «Запеченный лава с курицей»

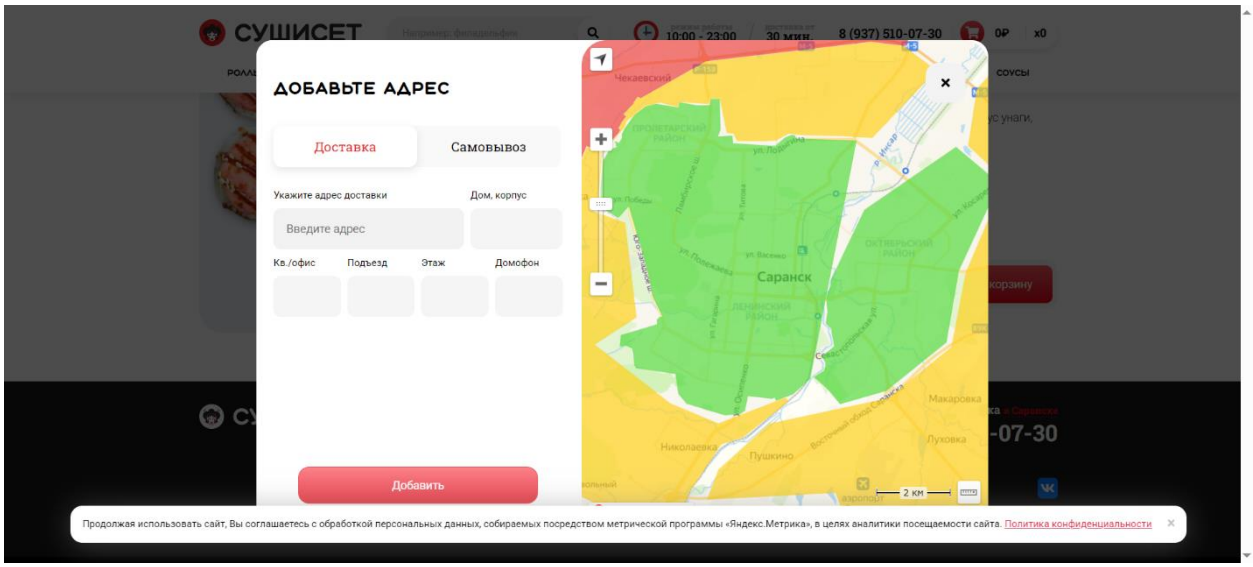

Рисунок 10 – Модальное окно с выбором адреса доставки после клика по кнопке «В корзину»

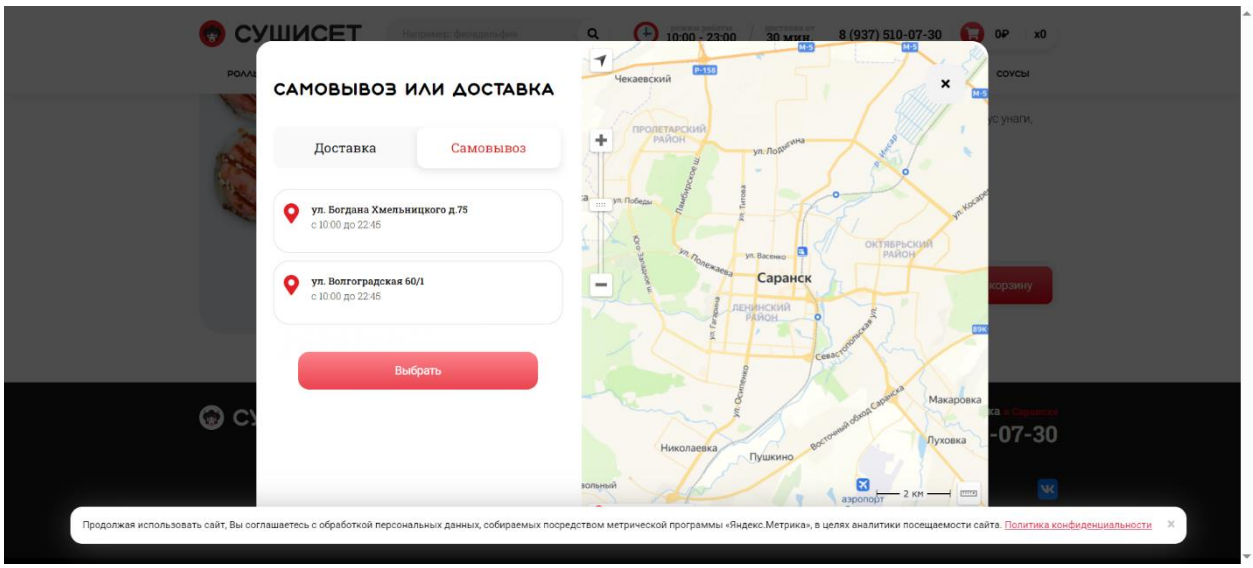

Рисунок 11 – Переход во вкладку «Самовывоз»

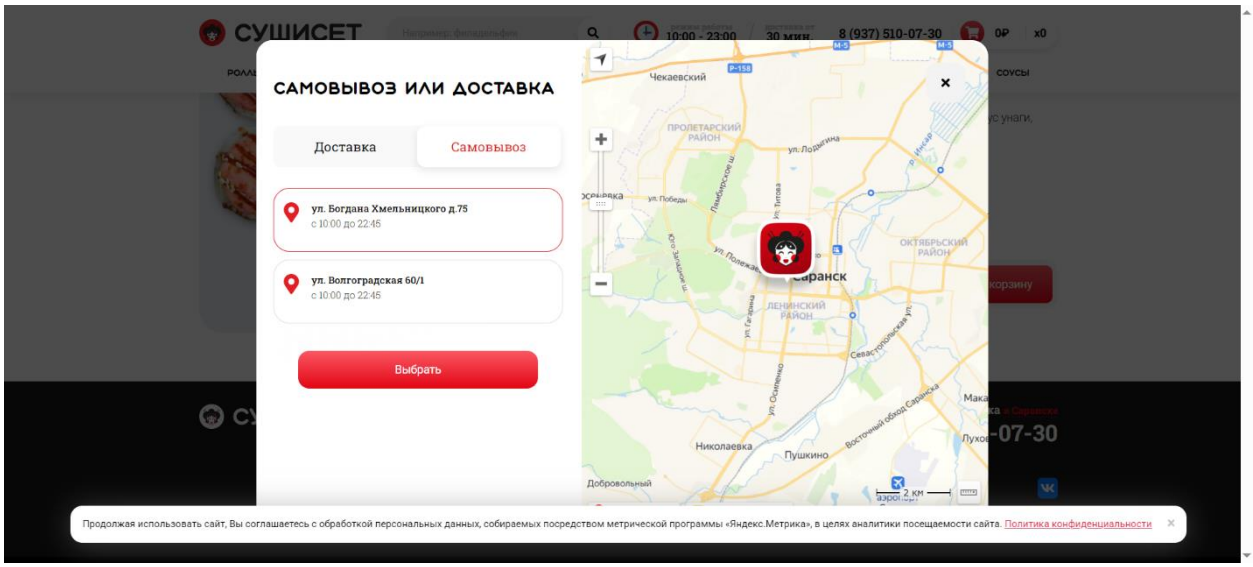

Рисунок 12 – Выбор первого из доступных адресов самовывоза

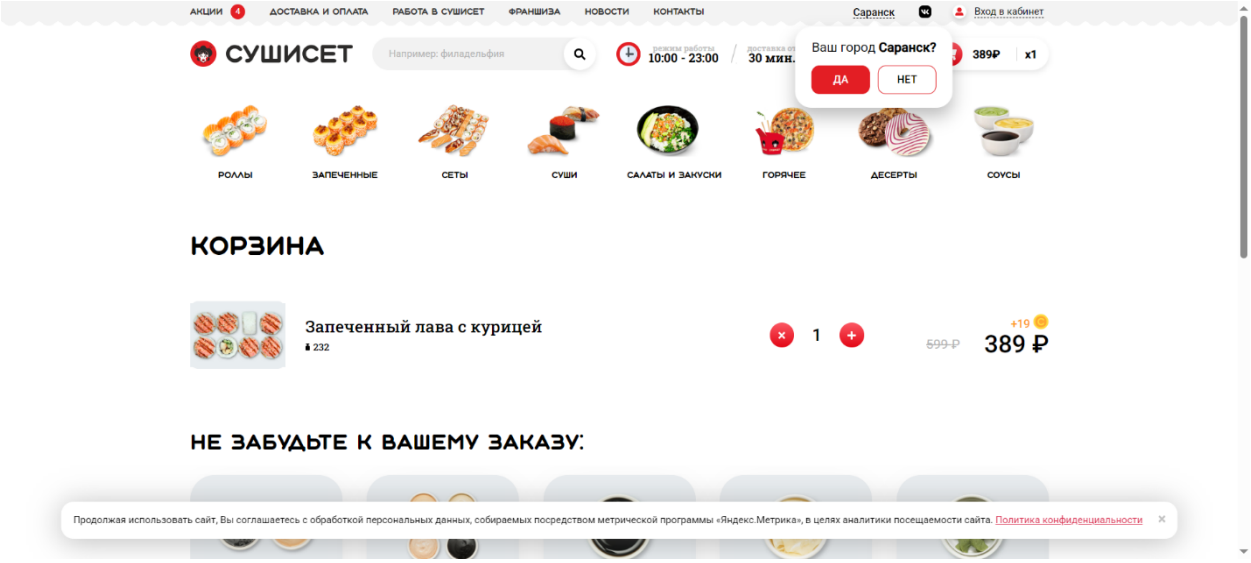

Рисунок 13 – Переход в корзину с добавленным товаром

Фактический результат полностью совпал с ожидаемым: товар появился в корзине с корректно указанным названием, ценой и количеством.

Негативный сценарий №1: поиск товара с некорректным названием.

На рисунках 14 – 15 представлены результаты автоматического тестирования позитивного кейса №1.

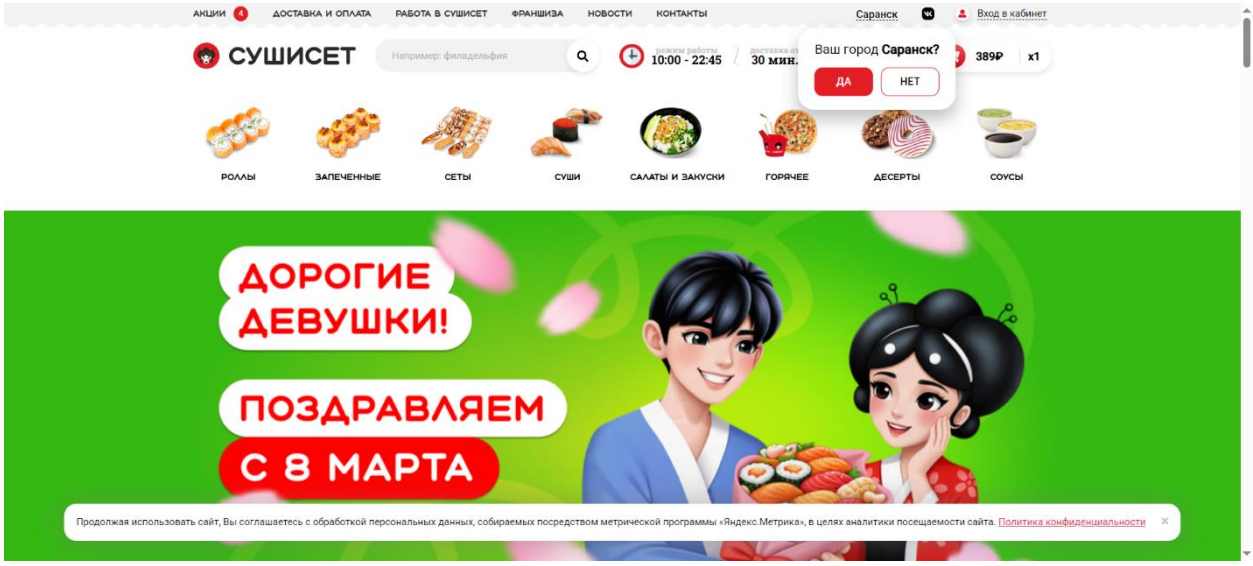

Рисунок 14 – Открыта главная страница сайта

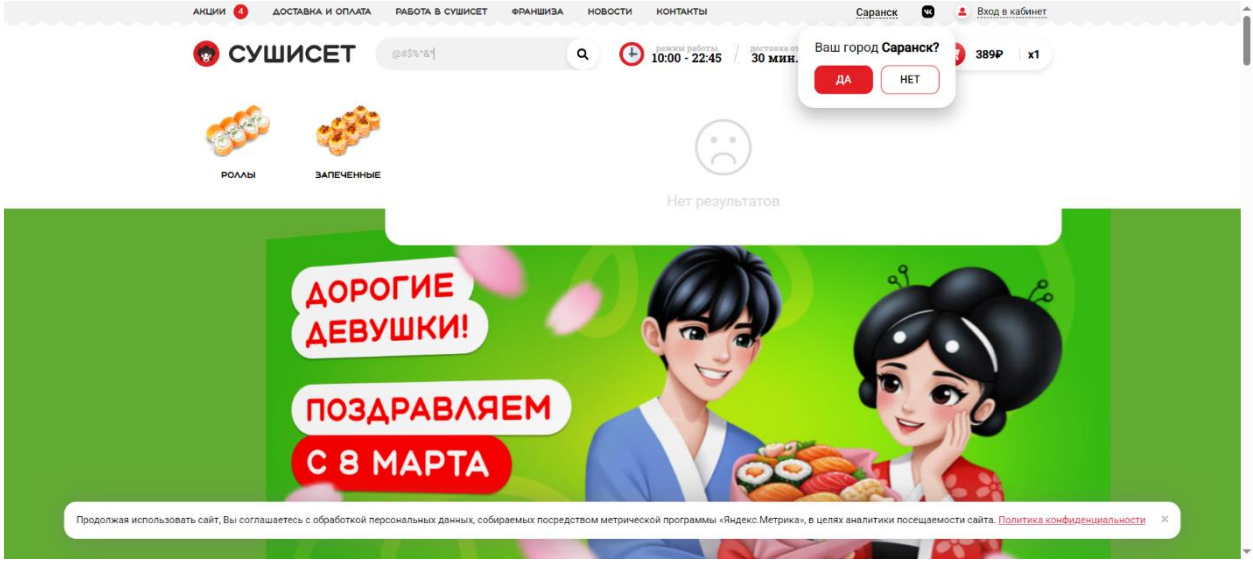

Рисунок 15 – Ввод в поисковую строку символов «@#$%^&*»

Фактический результат полностью совпал с ожидаемым: в выпадающем списке с результатами поиска появилось сообщение: «Нет результатов». Товар не найден.